

Performance Comparison of HTTP and MQTT Protocols for IoT Applications

Aram A. Ennab, Aya J. Abuzahrah, Manar A. Hamdan, Mayar A. Hamdan, Noor E. Alshater

Course: Scientific Research Methods In ICT

Instructor: Dr. Abdullah Al-Zaghameem

Date: January 6th 2026

Abstract

The Internet of Things (IoT) relies heavily on efficient communication protocols to enable reliable data exchange between interconnected devices. Among the widely used protocols in IoT environments are Hypertext Transfer Protocol (HTTP) and Message Queuing Telemetry Transport (MQTT), each offering distinct architectural and performance characteristics. This study presents a comparative analysis of HTTP and MQTT based on performance metrics including latency, scalability, throughput, and reliability under packet loss conditions. The evaluation is conducted using an experimental setup that simulates an IoT messaging environment with varying numbers of clients. The results, as demonstrated in the accompanying presentation, indicate that MQTT achieves lower latency, higher throughput, and improved reliability compared to HTTP, particularly as the number of connected clients increases. These findings highlight the suitability of MQTT for lightweight and scalable IoT applications, while HTTP remains more appropriate for traditional web-based communication scenarios.

Keywords

Internet of Things (IoT), HTTP, MQTT, Communication Protocols, Performance Evaluation, Latency, Throughput, Scalability, Reliability .

1. Introduction

The Internet of Things (IoT) represents a paradigm in which physical devices are interconnected through the internet to enable remote monitoring, control, and data exchange. IoT systems commonly consist of multiple sensor nodes, actuators, and control units that communicate with local gateways or remote servers through network infrastructures such as Wi-Fi or cellular networks. To support such communication, the selection of an appropriate application-layer protocol is a critical design consideration, as it directly affects system performance, scalability, and reliability [1].

Among the various communication protocols used in IoT environments, HTTP and MQTT are two widely adopted solutions. HTTP follows a request–response client–server architecture and was originally designed for web-based applications. While it provides high interoperability and simplicity, its relatively high overhead and latency may limit its efficiency in resource-constrained IoT systems. In contrast MQTT is a lightweight publish–subscribe protocol specifically designed for machine-to-machine communication. Its broker-based architecture, small message headers, and support for different Quality of Service (QoS) levels make it suitable for IoT scenarios that require efficient and reliable data delivery [1], [2].

This study focuses on comparing HTTP and MQTT in terms of latency, scalability, throughput, and reliability under controlled experimental conditions. The objective is to evaluate how each protocol behaves as the number of clients increases and to assess their suitability for IoT communication based on the observed performance results presented in the accompanying project presentation..

2. Overview of IoT Communication Protocols

2.1 MQTT Protocol (Message Queuing Telemetry Transport Protocol)

MQTT is a lightweight publish–subscribe messaging protocol specifically designed for machine-to-machine communication. It relies on a central broker that manages message distribution between publishers and subscribers, eliminating direct communication between clients. As illustrated in Figure 1, publishers send messages to the broker under specific topics, while subscribers receive messages by subscribing to those topics of interest. MQTT supports multiple Quality of Service (QoS) levels to ensure reliable message delivery and is characterized by small message headers and low bandwidth consumption, making it well suited for constrained IoT environments where efficient and scalable communication is required [2], [3], [4].

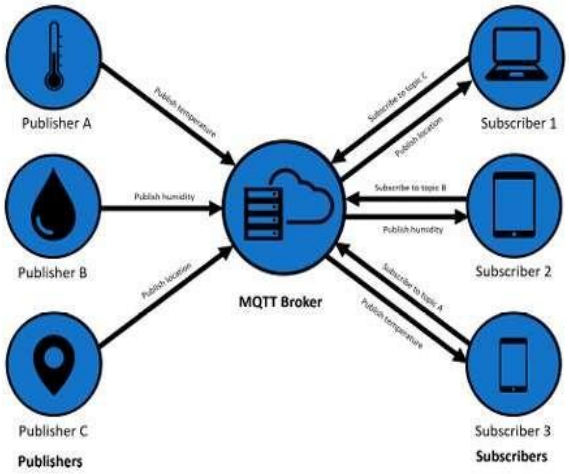


Figure 1. Publish/subscribe architecture in MQTT protocol.

2.2 HTTP (Hyper Text Transport Protocol)

HTTP is a widely used application-layer protocol based on a client–server request–response communication model. As illustrated in Figure 2, the client initiates communication by sending a request to the server, which processes the request and returns an appropriate response. HTTP is primarily designed for web-based communication and provides high interoperability across heterogeneous platforms. However, HTTP introduces higher protocol overhead and lacks built-in Quality of Service (QoS) mechanisms, which can negatively impact its performance in IoT systems where lightweight and efficient communication is required [1], [5].

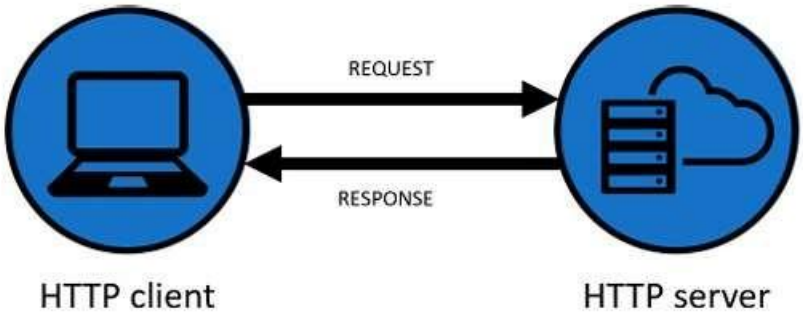


Figure 2. Request–response architecture

2.3 Comparative Analysis of HTTP and MQTT

To provide a clearer and structured comparison between the two communication protocols, a qualitative analysis of HTTP and MQTT is presented. This comparison focuses on key technical aspects that are particularly relevant to IoT environments, including communication architecture, message size, transport protocol, security mechanisms, encoding format, Quality of Service (QoS), reliability, and organizational support [1], [2], [3].

Table 1. Comparison between HTTP and MQTT Protocols

Criteria	MQTT	HTTP
Architecture	Client/Broker	Client/Server
MessageSize	Small andUndefined(up to 256 MB maximum size)	Large and Undefined (depends on the web server or the programmingtechnology)
Transport Protocol	TCP(MQTT-SNcanuse UDP)	TCP
Security	TLS/SSL	TLS/SSL
Encoding Format	Binary	Text

Quality of Service (QoS)/ Reliability	QoS 0- At most once (Fireand-Forget), QoS1-At leastonce, QoS2-Exactlyonce	Limited (via Transport Protocol-TCP)
Organisational Support	IBM, Facebook, Eurotech, Cisco, Red Hat,SoftwareAG,Tibco, ITSO, M2Mi, Amazon Web Services (AWS), InduSoft,Fiorano	Global Web Protocol Standard

As summarized in Table 1, MQTT and HTTP differ significantly in terms of architectural design and communication efficiency. MQTT adopts a client–broker architecture with a publish–subscribe communication model, enabling asynchronous message delivery and reduced coupling between communicating entities. In contrast, HTTP relies on a traditional client–server request–response model, which can introduce higher latency and overhead in large-scale IoT deployments [3], [6].

2.4 Performance Comparison Between HTTP and MQTT

2.4.1 Bandwidth and Latency Analysis

HTTP generally exhibits higher bandwidth consumption and increased latency compared to lightweight IoT protocols. This behavior is mainly attributed to its reliance on TCP, which introduces connection establishment overhead and employs a slow-start mechanism that limits bandwidth utilization during initial transmissions. As a result, HTTP often experiences longer response times, making it less suitable for latency-sensitive IoT applications [1], [2].

MQTT also operates over TCP and therefore inherits similar transport-layer limitations. However, due to its lightweight design and compact message headers, MQTT typically achieves lower latency than HTTP. Higher Quality of Service (QoS) levels may increase bandwidth consumption because of additional handshake mechanisms, but MQTT generally maintains better performance in IoT-oriented deployments [2], [6].

2.4.2 Reliability and Quality of Service (QoS) vs. Interoperability

HTTP was primarily designed to ensure high interoperability across the Web rather than to provide native reliability mechanisms. Consequently, its reliability depends mainly on TCP guarantees, as HTTP itself does not support explicit Quality of Service control.

Despite this limitation, HTTP-based systems remain highly interoperable, requiring only an HTTP protocol stack for communication.

MQTT provides explicit reliability control through multiple Quality of Service levels, including at-most-once, at-least-once, and exactly-once delivery. These mechanisms allow MQTT to better support reliability-critical IoT applications [3], [6]. However, its publish–subscribe communication model may offer lower interoperability compared to HTTP due to its specialized architecture

2.4.3 Message Size and Protocol Overhead

Although both MQTT and HTTP operate over TCP and incur similar connectionlevel overhead, MQTT is designed to minimize protocol overhead through compact binary headers, which can be as small as two bytes [4],This lightweight structure reduces overall message size and improves efficiency in constrained environments.

HTTP, on the other hand, introduces significant overhead due to verbose textbased headers and its original design for web communication rather than IoT systems. Consequently, HTTP typically incurs larger message sizes and higher overhead, which can negatively impact performance in bandwidth-limited IoT deployments [3], [2].

2.5 Security Implications of MQTT and HTTP

Security is a critical concern in Internet of Things (IoT) environments, as the interconnection of multiple devices—often deployed and managed by non-expert users— introduces significant security risks. Many commonly used IoT communication protocols do not provide built-in security mechanisms by default. While security measures enhance data protection, they may introduce additional computational and communication overhead, potentially affecting system performance.

MQTT, although widely adopted as a lightweight publish–subscribe protocol, does not inherently provide security, as it operates over TCP. To secure MQTT communications, Transport Layer Security (TLS) can be employed to ensure authentication, data confidentiality, and integrity. However, the use of TLS introduces additional memory and processing overhead, which may be challenging for resource-constrained IoT devices. In such cases, alternative approaches, including payload encryption and secure credential handling within CONNECT messages, can be considered. Additionally, TLS-PSK cipher suites may reduce computational complexity by eliminating public-key operations, although their adoption remains limited and dependent on library support [7].

3. Experimental Setup and Methodology

This section describes the experimental setup, communication models, network conditions, data collection process, and statistical methods used to evaluate the performance of HTTP and MQTT in an IoT-like environment.

3.1 Experimental Environment

The experimental setup was designed to compare the performance of HTTP and MQTT under controlled and reproducible conditions. All experiments were conducted on an Ubuntu Server virtual machine running on Oracle VirtualBox. Python 3 was used to implement the client and server applications for both protocols to ensure consistency across all test scenarios.

For MQTT evaluation, a local MQTT broker was deployed. For HTTP evaluation, a Flaskbased HTTP server was implemented. All experiments were executed within the same virtualized environment to eliminate hardware-related performance variations.

3.2 Communication Models

This study evaluates two different communication models commonly used in IoT systems: the publish–subscribe model and the request–response model.

3.2.1 Publish–Subscribe Model (MQTT)

The publish–subscribe communication model was evaluated using MQTT A subscriber application was first executed to subscribe to a predefined topic and continuously wait for incoming messages. Subsequently, a publisher application transmitted a sequence of messages to the MQTT broker [3], [4].

Latency measurements were obtained by recording timestamps at the time of message publication and reception. Messages were transmitted sequentially, without parallel execution, to avoid timing distortion. A single Quality of Service level (QoS 1) was enforced throughout the experiments to maintain consistent reliability conditions [4].

3.2.2 Request–Response Model (HTTP)

The request–response communication model was evaluated using HTTP. A Flaskbased HTTP server was launched locally and configured to provide a fixed endpoint that accepts HTTP POST requests. The client application sent a sequence of requests to the server and measured round-trip latency based on the time difference between request transmission and response reception [1], [5].

All HTTP requests were processed sequentially, and server-side logs were used to confirm successful request handling through HTTP 200 response codes. This approach ensured fair comparability with the MQTT experiments [5].

3.3 Test Parameters and Payload Configuration

To ensure a fair comparison between the two protocols, identical payload formats and message sizes were used. Messages were transmitted using a fixed payload size, and the total number of messages was kept constant across both protocols. Client scaling experiments were conducted by gradually increasing the number of concurrent clients while monitoring protocol performance.

3.4 Network Impairment and Packet Loss Simulation

To assess protocol reliability under adverse network conditions, packet loss was introduced using Linux traffic control tools. Network emulation was performed using the netem module [7]to simulate a lossy network environment with a predefined packet loss rate. Reliability was measured based on the successful delivery of messages within a specified timeout period, allowing a controlled comparison of HTTP and MQTT behavior under packet loss conditions.

3.5 Performance Metrics

Protocol performance was evaluated using the following metrics:

Latency: Measured as the round-trip time for message delivery.

Throughput: Calculated as the number of successfully delivered messages per second.

Reliability: Evaluated based on message delivery success rate under packet loss conditions.

Scalability: Assessed by increasing the number of concurrent clients and observing latency behavior.

3.6 Data Collection Procedure

During the experiments, performance data were collected automatically using Pythonbased scripts. Latency values were recorded for each transmitted message, throughput was computed over fixed time intervals, and reliability data were logged during packet loss simulations. All raw measurements were stored for subsequent analysis and visualization.

4. Results and Statistical Analysis

This section presents the experimental results obtained from the performance evaluation of HTTP and MQTT. The analysis focuses on scalability, correlation behavior, throughput, and reliability. The corresponding figures referenced in this section are provided in the accompanying project presentation.

4.1 Scalability Analysis (Latency vs Number of Clients)

Figure 1, as shown in the accompanying project presentation, illustrates the impact of increasing the number of clients on latency for both HTTP and MQTT protocols. The experimental results indicate that the latency of the HTTP protocol increases significantly as the number of clients grow, rising from 90 ms at 10 clients to 285 ms at 50 clients. This sharp increase reflects the sensitivity of HTTP to higher numbers of concurrent connections.

In contrast, MQTT demonstrates a limited and gradual increase in latency, rising from 33 ms to only 48 ms within the same client range. These observations indicate that MQTT exhibits better scalability characteristics, as it maintains more stable latency under higher loads compared to HTTP

4.2 Correlation Analysis Between Number of Clients and Latency

A correlation analysis was conducted to examine the relationship between the number of connected clients and latency for both protocols. The correlation values were computed based on the experimental data presented in the project presentation.

The results reveal a very strong positive correlation between the number of clients and latency for both protocols. Specifically, the correlation coefficient for HTTP was ($r = 0.983$), while MQTT achieved a correlation coefficient of ($r = 0.993$). These values indicate that latency increases as the number of clients increase, with HTTP exhibiting a higher rate of performance degradation compared to MQTT.

4.3 Average Throughput Comparison

Figure 2, as provided in the project presentation, presents a comparison of the average throughput achieved by HTTP and MQTT. The results show that MQTT achieves a higher average throughput of 262.3 messages per second, whereas HTTP achieves an average throughput of 115.3 messages per second under identical testing conditions.

This difference highlights the efficiency of MQTT in handling message transmission, particularly in environments that require lightweight communication and support for a large number of concurrent messages.

4.4 Reliability Analysis (Packet Loss)

Figure 3 illustrates the comparison of average packet loss between HTTP and MQTT under unstable network conditions, as shown in the project presentation. The experimental findings indicate that the average packet loss for HTTP reached 5.19%, while MQTT experienced a significantly lower packet loss rate of 1.02%.

These results suggest that MQTT provides improved reliability compared to HTTP when operating in lossy network environments, making it more suitable for IoT applications that require dependable message delivery.

4.5 Results Summary

Based on the experimental results presented, MQTT demonstrates superior performance compared to HTTP in terms of scalability, throughput, and reliability. The observed measurements indicate that MQTT is more suitable for applications that require efficient communication and stable performance as the number of connected clients increases, while HTTP remains more appropriate for traditional web-based communication scenarios.

5. Conclusion

This study presented a comparative evaluation of HTTP and MQTT protocols for Internet of Things (IoT) applications, focusing on key performance metrics including latency, scalability, throughput, and reliability under packet loss conditions. Through a controlled experimental setup, the behavior of both protocols was analyzed as the number of connected clients increased.

The experimental results demonstrate that MQTT consistently outperforms HTTP in terms of scalability, achieving lower latency and higher throughput while maintaining improved reliability in lossy network environments. These characteristics make MQTT particularly suitable for IoT systems that require lightweight communication, efficient resource utilization, and stable performance under increasing load [2], [3], [7]. In contrast, HTTP remains a viable option for traditional web-based applications where interoperability and simplicity are prioritized over performance efficiency.

Overall, the findings of this study highlight the importance of selecting communication protocols based on application requirements. Future work may extend this comparison by evaluating additional protocols, such as CoAP, or by incorporating security overhead measurements to further assess performance trade-offs in real-world IoT deployments.

References

[1] V. Karagiannis, P. Chatzimisios, F. Vazquez-Gallego and J. Alonso-Zarate, " A survey on application layer protocols for the Internet of Things," *Transactions on IoT and Cloud Computing,* vol. 3, no. 1, p. 11–17, 2015.

[2] B. V. Wukkadada, K. Wankhede, R. Nambiar and A. Nair, " Comparison of HTTP and MQTT protocols in Internet of Things (IoT)," in *International Conference on Inventive Research in Computing Applications (ICIRCA 2018)*, 2018.

[3] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *IEEE International Systems Engineering Symposium (ISSE)*, 2017.

[4] A. Banks and R. Gupta, " MQTT Version 3.1.1," OASIS Standard, 2014.

[5] R. T. Fielding, M. Nottingham and J. Reschke, " Hypertext Transfer Protocol (HTTP) Semantics," IETF RFC 9110, 2022.

[6] D. Thangavel, X. Ma, A. Valera, H. X. Tan and C. K. Y. Tan, " Performance evaluation of MQTT and CoAP via a common middleware," in *IEEE ISSNIP Conference*, 2014.

[7] T. Prantl, L. Iffländer, S. Herrnleben, S. Engel, S. Kounev and C. Krupitzer, " Performance impact analysis of securing MQTT using TLS," in *ACM/SPEC International Conference on Performance Engineering (ICPE '21)*, 2021.